Análisis de Caso

Representación de una Arquitectura Cloud Orientada a Microservicios

Eva Durán

alkemy

Análisis de Caso

Representación de una Arquitectura Cloud Orientada a Microservicios

Situación inicial 📍

Una empresa emergente llamada **CloudServicesCo** quiere modernizar su infraestructura para mejorar la escalabilidad y disponibilidad de sus aplicaciones. Hasta ahora, la empresa contaba con un sistema monolítico alojado en servidores locales, lo que generaba frecuentes problemas de lentitud y dificultades para implementar nuevas funcionalidades. El equipo de TI ha decidido migrar a la nube de AWS y adoptar una **arquitectura de microservicios** para dividir la aplicación en partes más manejables.

Sin embargo, antes de avanzar con la migración real, necesitan **representar y validar visualmente** cómo se vería la arquitectura en AWS. De esta manera, podrán identificar componentes, interconexiones, patrones de seguridad y costos aproximados, reduciendo riesgos en etapas posteriores de implementación.

Descripción del Caso 🔍

Eres un consultor en la empresa CloudServicesCo y tu misión es **diseñar un diagrama de arquitectura** que represente de forma clara y concisa la nueva propuesta. Este diagrama debe mostrar al menos tres microservicios principales (por ejemplo, "Autenticación", "Catálogo" y "Pedidos"), así como los servicios de AWS que se interconectan para manejar networking, balanceo de carga y base de datos.

El objetivo es que, al finalizar, el equipo directivo y los desarrolladores tengan **una visión clara** de cómo se interrelacionan los componentes en la nube, qué servicios de AWS se utilizarán y cuál será el flujo de datos entre ellos.

1. Identificar Servicios Clave en AWS Academy

1) Amazon ECS (Fargate) — Cómputo de los microservicios

Rol: ejecutar los contenedores de Autenticación, Catálogo y Pedidos sin gestionar servidores.

Por qué: simplifica operación vs. EC2. Auto Scaling por métricas.

Uso: cada microservicio corre como un "ECS" con su Task Definition independiente.

2) Application Load Balancer (ELB/ALB) — Entrada y enrutamiento

Rol: redirigir el tráfico entrante HTTP/HTTPS y distribuirlo hacia cada microservicio (Autenticación, Catálogo, Pedidos) mediante reglas de enrutamiento.

Por qué: integración nativa con ECS; soporta TLS, WAF y health checks por target group.

Uso: un ALB público con punto de entrada 443 (HTTPS) que escuchar solicitudes y las distribuye a los microservicios.

3) Amazon RDS (PostgreSQL) — Base de datos transaccional

Rol: persistencia relacional (usuarios, productos, pedidos).

Por qué: Multi-AZ, backups automáticos, performance estable y SQL estándar; reduce carga operativa.

Uso: subredes privadas/aisladas; conexión TLS desde los contenedores.

4) Amazon S3 — Almacenamiento de objetos

Rol: imágenes de producto, adjuntos, artefactos y respaldos lógicos.

Por qué: costos bajos por GB; versionado y políticas de ciclo de vida.

Uso: acceso privado vía VPC Endpoint; opcional CDN si luego se quiere agregar distribución pública.

5) Amazon SQS — Mensajería asíncrona

Rol: cola de tareas de "Pedidos" (confirmaciones, post-procesos,

notificaciones) para desacoplar y absorber picos.

Por qué: aporta resiliencia al sistema, permite reintentos automáticos, desacopla la API de Pedidos de los procesos más pesados y facilita absorber picos de carga sin comprometer la disponibilidad de la aplicación.

Uso: La API de pedidos recibe las solicitudes de los usuarios (como cuando alguien hace una compra) y envía un mensaje a la cola (SQS) en lugar de procesarlo todo en ese momento. Los servicios de procesamiento toman la orden de la cola y la completan en la base de datos.

Complementos recomendados

- Amazon Cognito: registro/login. El microservicio Autenticación valida y emite/gestiona tokens digitales.

- Amazon CloudWatch: métricas, logs y alarmas por microservicio.

- AWS WAF: firewall de aplicaciones web que protege contra ataques comunes y evita abusos con límites de solicitudes (rate limiting).

- AWS Secrets Manager: servicio que guarda y rota automáticamente credenciales y llaves de acceso, evitando exponerlas en el código o en la configuración.

- Amazon VPC (2 AZ, zonas de disponibilidad) y Subredes: red privada virtual que organiza los recursos en subredes públicas (ALB/NAT), privadas (ECS) y aisladas (RDS), usando VPC Endpoints para un acceso seguro a servicios como S3 y CloudWatch.

2. Diseño del diagrama de arquitectura

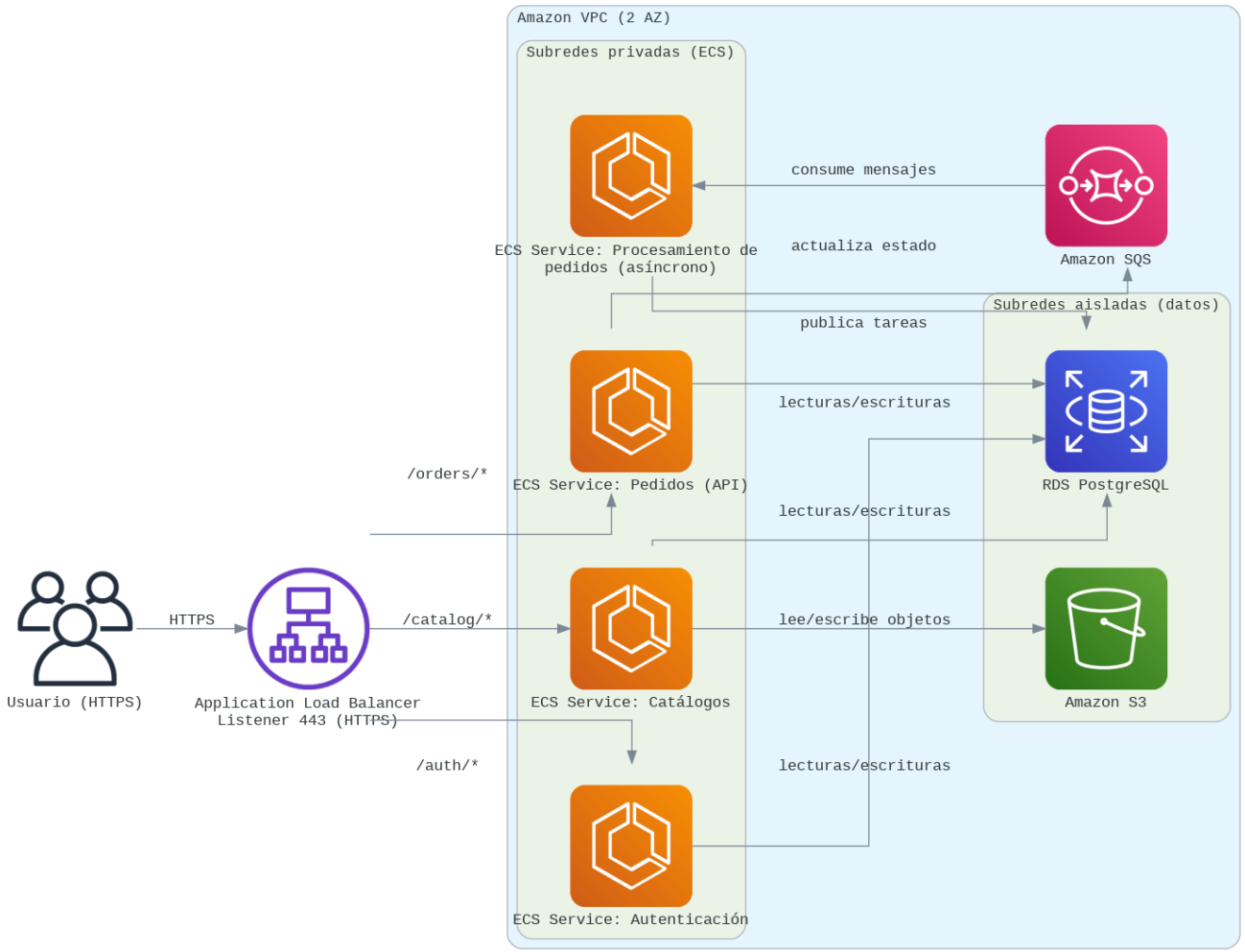

Figura 1. Propuesta de diagrama de arquitectura cloud orientada a microservicios para la empresa CloudServicesCo.

3. Flujo de datos

1) El Usuario accede por HTTPS (conexión cifrada) y llega al ALB (puerto 443).

2) El ALB enruta según la ruta de la URL:

- /auth/* → ECS: Autenticación
- /catalog/* → ECS: Catálogo
- /orders/* → ECS: Pedidos (API)

3) Autenticación verifica credenciales. Si son válidas, emite un token de sesión (JWT: JSON Web Token) que acompañará las siguientes solicitudes.

4) Catálogo atiende búsquedas y listados: realiza lecturas en RDS (productos, precios, stock) y lee/escribe archivos en S3 (imágenes y adjuntos).

5) Pedidos (API) recibe la creación/consulta/cancelación de pedidos: escribe y lee en RDS (órdenes, estados, totales).

6) Para operaciones que pueden tardar (confirmaciones, actualización de stock, notificaciones), Pedidos (API) publica un mensaje en SQS (no bloquea al usuario).

7) El Servicio de procesamiento de pedidos (asíncrono, en ECS) consume los mensajes de SQS y ejecuta las tareas pendientes:

- Actualiza el estado del pedido y el stock en RDS.
- Opcionalmente genera registros o notificaciones (no visibles en este diagrama).

8) Si el consumo falla, SQS permite reintentos; el mensaje permanece en cola hasta procesarse (resiliencia).

9) Todas las comunicaciones internas entre microservicios y datos se realizan dentro de la VPC (red privada) y las conexiones a RDS se hacen con cifrado en tránsito.

La representación visual de la arquitectura permite identificar de manera temprana posibles cuellos de botella en el flujo de datos, como dependencias excesivas en la base de datos relacional o puntos únicos de entrada a través del balanceador de carga. Al ver claramente cómo se conectan los microservicios y los componentes de almacenamiento y mensajería, el equipo puede anticipar dónde podrían generarse sobrecargas y planificar mecanismos de escalado o cacheo antes de que ocurran problemas en producción.

Además, el diagrama funciona como una herramienta de comunicación común para todo el equipo. Facilita que perfiles técnicos y no técnicos comprendan cómo fluyen las solicitudes, qué rol cumple cada servicio y cómo se aseguran atributos como escalabilidad o resiliencia. Esta claridad ayuda a alinear decisiones y reduce malentendidos. Por último, al representar explícitamente servicios administrados como RDS, S3 o SQS, se pueden anticipar los costos asociados (cómputo, almacenamiento, tráfico de red) y evaluar alternativas de optimización, equilibrando así el diseño técnico con la administración financiera del proyecto.